

NDTwin Network Digital Twin

P4 / bmv2 Data-Plane Support

— a second data plane, with nothing above it changed

Outline

- | | | |
|---|---------------------------------------|-----------|
| 0 | Background | pp. 4–9 |
| The system I inherited, the architecture, the stack, and the scale of what changed. | | |
| 1 | New capabilities | pp. 11–18 |
| The P4 pipeline, the proxy agent, telemetry synthesis, liveness, failover, power. | | |
| 2 | Test tooling and documentation | pp. 20–23 |
| The methodology, the five-layer harness, and what was written down. | | |
| 3 | Measured results | pp. 25–40 |
| Differential testing, failover, throughput, load, twin accuracy — and a live demo. | | |

SECTION 0

Background

The system I inherited, and where the P4 path attaches to it

Background: the system, and the task

NDTwin is a network digital twin — a live model of a running network, kept in step with it

1 A C++ kernel holds the model

It owns the topology graph, ingests sFlow telemetry, tracks flows and link usage, and answers queries about the live network.

2 Seven components depend on it

Every one of them reaches the kernel only through the /ndt/ HTTP API — no shared memory, no shared files, no other channel.

3 The existing system drove OVS

Mininet with Open vSwitch, controlled by Ryu. Built by the lab before me; treated here as the given starting point.

4 My task

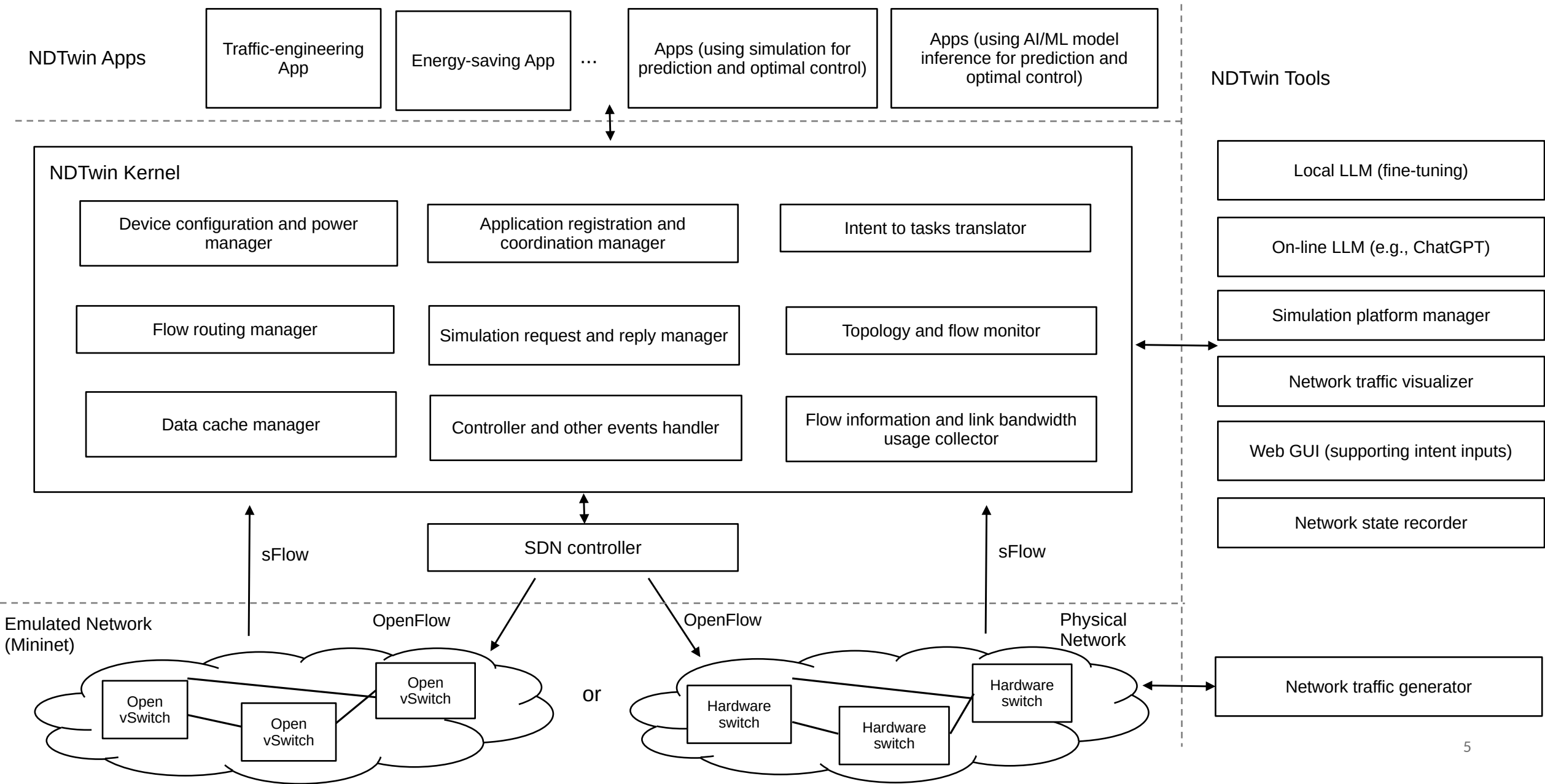
Make the same twin drive a P4 / bmv2 data plane as well — without the seven components having to change.

The seven consumers

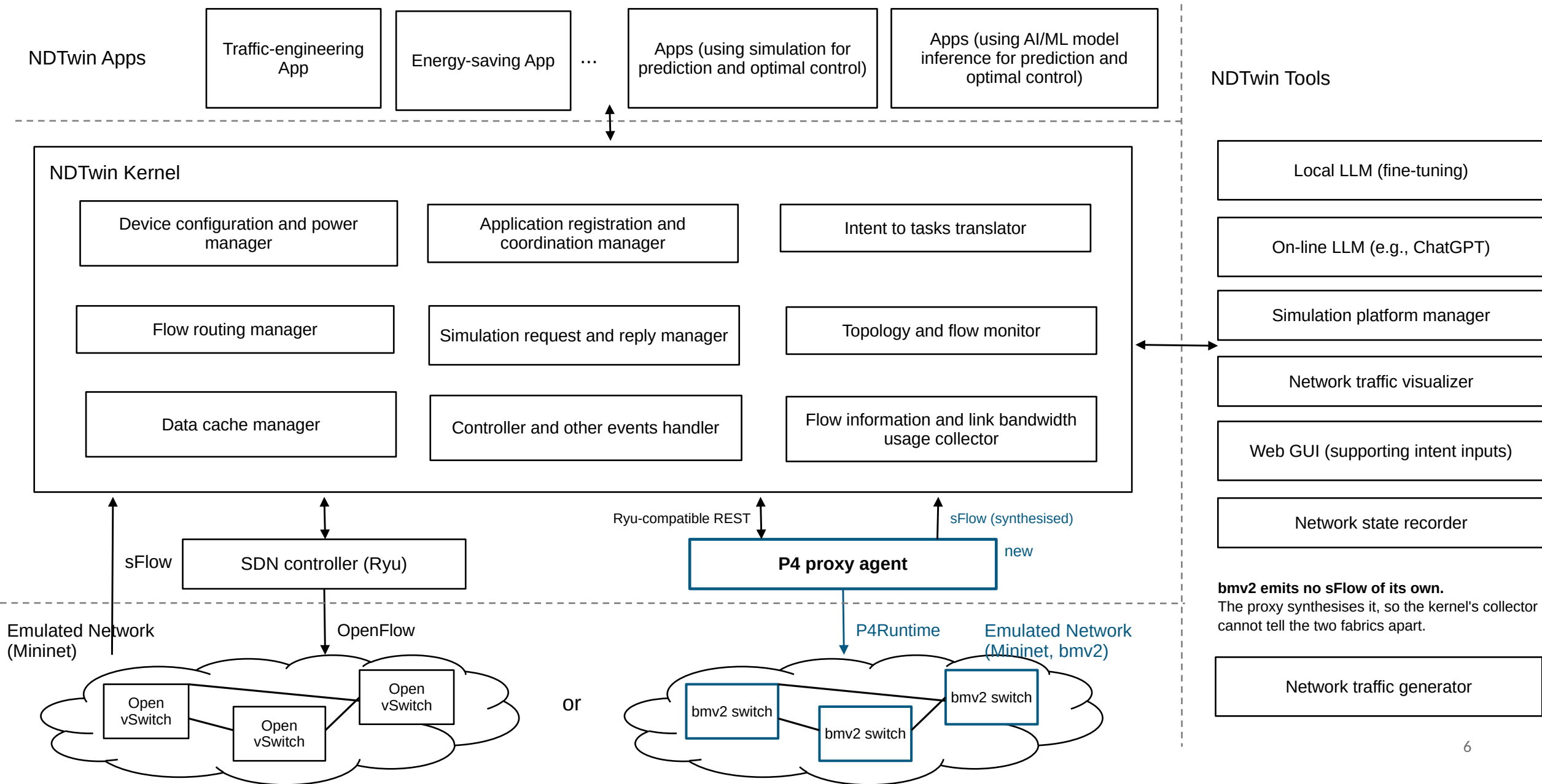
Energy-Saving-App	C++
Traffic-Engineering-App	Python
Web-GUI	React / Node
Network-Traffic-Visualizer	JavaFX
Network-State-Recorder	Python
Network-Traffic-Generator	Python
Simulation-Platform-Manager	C++

All seven read the graph through /ndt/get_graph_data. If that endpoint breaks, all seven break at once.

NDTwin: A Framework for Network Digital Twins



NDTwin: A Framework for Network Digital Twins



Architecture: how P4 attaches to the existing system

The proxy agent impersonates Ryu's northbound API — nothing above the kernel changes

1 The proxy speaks Ryu, not the kernel

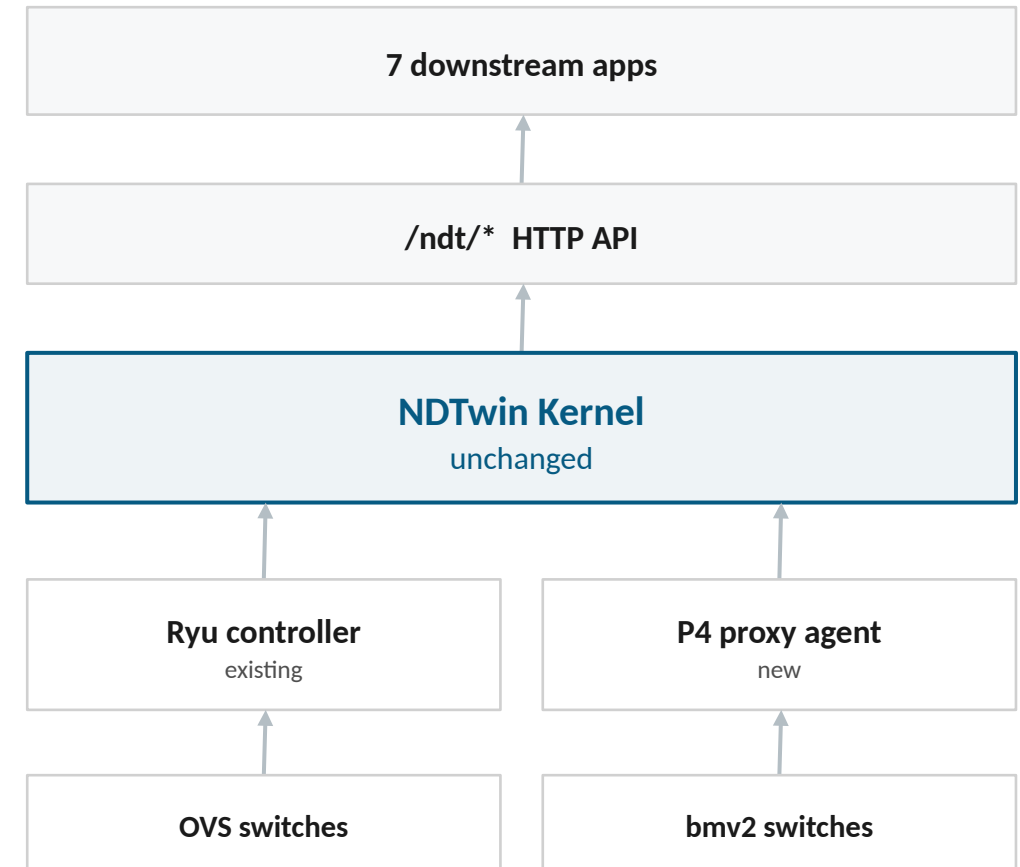
Topology, /stats/flow and destination paths are returned in Ryu's data shapes, so the kernel issues exactly the requests it has always issued.

2 sFlow v5 synthesised in the proxy

bmv2's sampled packets are synthesised into sFlow v5 and sent to the kernel's existing UDP:6343 collector, byte-layout compatible with OVS.

3 Result: zero changes above the kernel

Classifier, FlowLinkUsageCollector, every /ndt/ endpoint, the Intent Translator and all 7 downstream apps are untouched.



The stack

What the two data planes and the layer between them are built out of

Kernel

C++23

CMake

build, with opt-in sanitizer targets

Boost.Beast / Asio / URL

the /ndt/ HTTP server

nlohmann::json

every request and reply

spdlog

logging, with a level-gated trap of its own

libssh

southbound to hardware switches

GTest

603 cases

ASan / TSan

reproducible sanitizer builds

P4 data plane

P4_16

v1model architecture

the target the pipeline compiles for

bmv2 simple_switch_grpc

the software switch

p4c-bm2-ss

the compiler

P4Runtime

gRPC + protobuf control API

PRE clone session

how telemetry samples reach the CPU

Mininet

the fabric the switches run in

Proxy and control plane

Python

FastAPI / uvicorn

the proxy's north side

grpcio

the proxy's south side

networkx

graph state and BFS routing

Ryu / OpenFlow 1.3

the API being impersonated

sFlow v5

synthesised, not forwarded

LLDP

link discovery and liveness beacons

hypothesis

property-based topology tests

Scale of the work

377 commits, 2026-07-23 → 2026-08-19, on top of baseline 28b8b13

	lines added		files	removed
Documentation		+46,290	207	-5
Runbooks, design and status docs, investigation records				
Tests & tooling		+25,148	117	—
gtest suites, P4-proxy Python tests, the L0–L4 harness				
Kernel (C++)		+6,068	56	-1,053
39 existing files modified, 17 new				
P4 proxy		+4,431	26	—
All new — P4Runtime client, REST shim, sFlow emitter, P4 pipeline				
Total		+81,937	406	-1,058

Diffed against baseline 28b8b13. Comment lines, blank lines and build artefacts are excluded; intelligent_router.py is excluded as the lab's pre-existing Ryu controller.

SECTION 1

New capabilities

The P4 pipeline, the proxy agent, telemetry synthesis, liveness, failover, power

The P4 pipeline

Phase 4 — ndtwin_switch.p4, extended from a forwarding table into something that produces the same signal OVS does

1 A ternary 5-tuple table, ahead of LPM

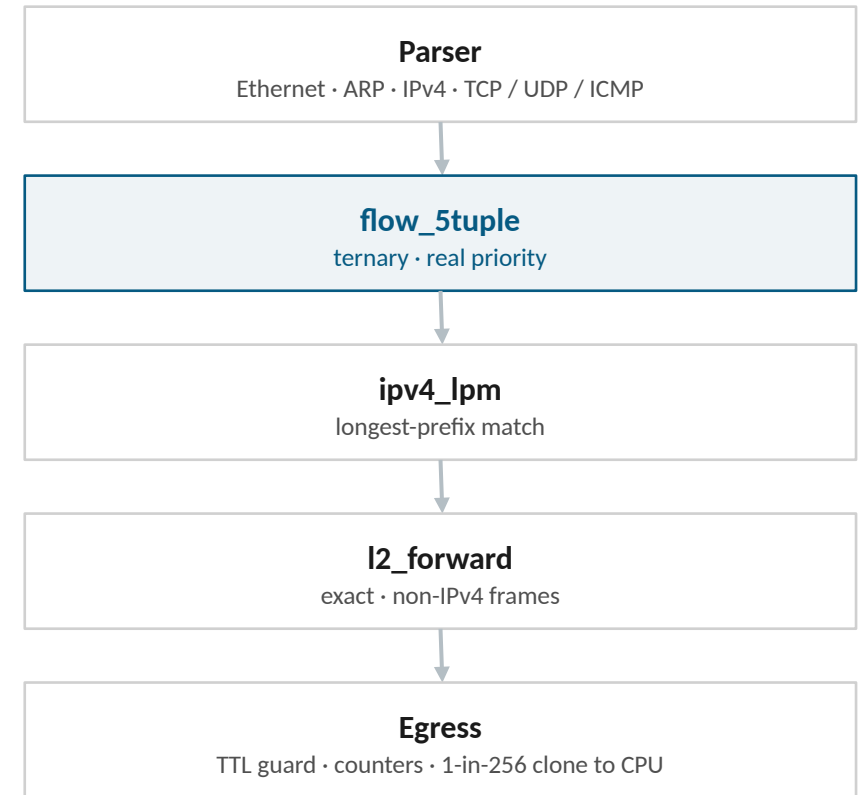
flow_5tuple carries a real priority and is evaluated before ipv4_lpm, so a specific flow rule wins over the default route rather than depending on table order.

2 Non-IPv4 frames are no longer dropped

ARP, TCP, UDP and ICMP are parsed, and an L2 table forwards what the IPv4 path cannot. Before this, anything that was not IPv4 disappeared silently.

3 Telemetry is generated in the pipeline

Direct and per-port counters, a TTL guard, and 1-in-256 clone-to-CPU sampling — the source of every flow sample the kernel later sees.



482 lines of P4_16 / v1model, compiled with p4c-bm2-ss.

Inside the pipeline

What a packet actually meets, in the order it meets it

Order is the design

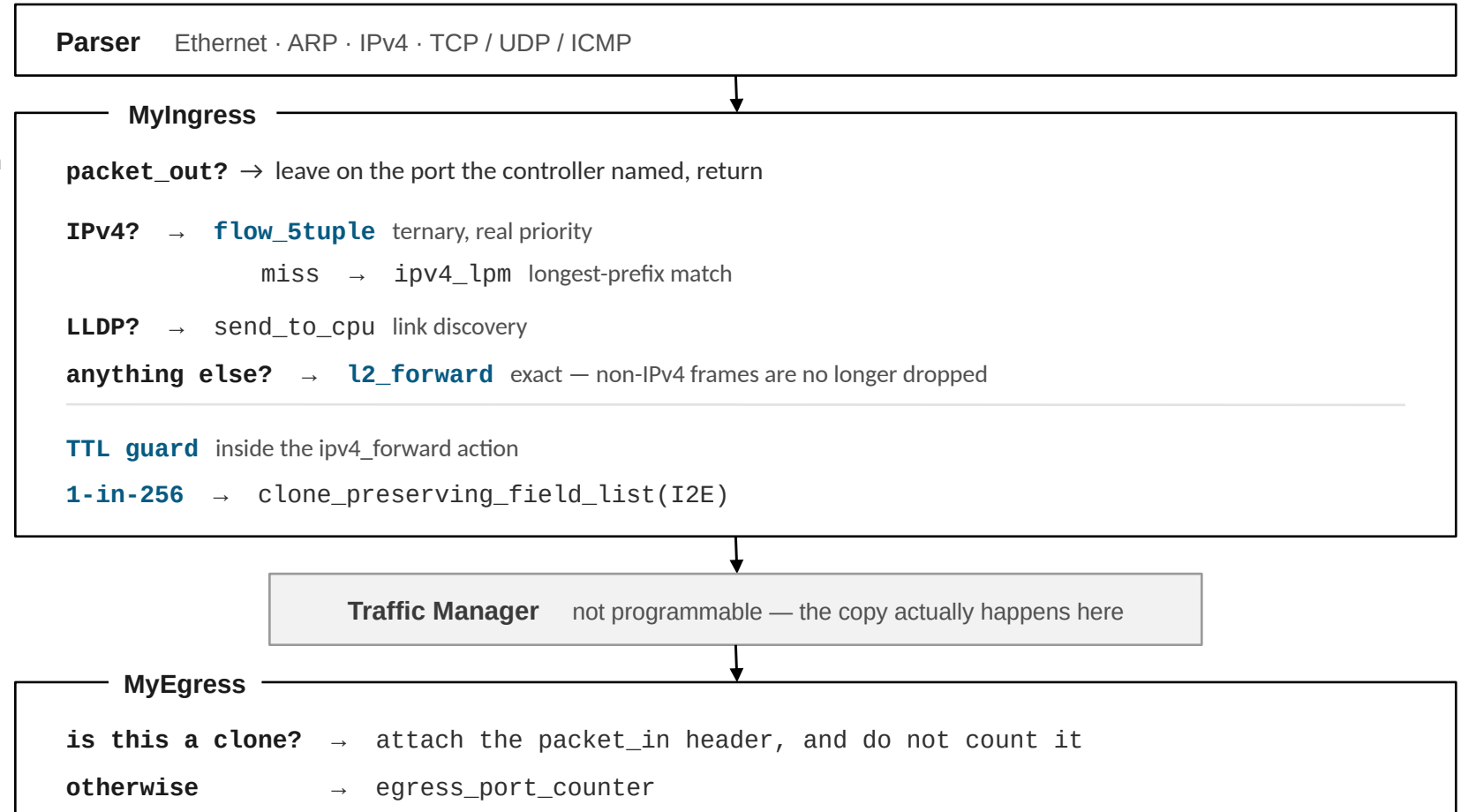
flow_5tuple is consulted before ipv4_lpm, so a specific rule beats the default route without depending on table order.

Two exits before routing

A packet_out from the controller leaves immediately; LLDP goes to the CPU. Neither reaches the forwarding tables.

Cloning is not ours

The ingress block only marks a packet. The Traffic Manager performs the copy, and that stage is not programmable.



The P4 proxy agent

A FastAPI service that answers in Ryu's shapes on the north side and speaks P4Runtime on the south

1 Ryu's northbound API, reimplemented

/v1.0/topology/*, /stats/flowentry/*, /stats/flow/<dpid> and destination paths, all returned in the exact shapes Ryu produces — including its string-action flow format.

2 P4Runtime on the south side

gRPC to bmv2's simple_switch_grpc. Switch connections are opened concurrently at startup, and every unary call carries a deadline so one dead switch cannot stall the rest.

3 Only /p4/ is new vocabulary

Where Ryu has no equivalent to impersonate — switch liveness — the endpoint is namespaced under /p4/ rather than pretending to be something Ryu offers.

Modules		lines
topology_manager.py	Graph state, BFS routing, LLDP beacons	1,392
p4_client.py	P4Runtime gRPC, clone session, packet-in split	587
sflow_emitter.py	sFlow v5 synthesis	448
ryu_topology.py	/v1.0/topology/* in Ryu's shapes	338
main.py	Startup, concurrent switch connect	305
api_routes.py	/stats/flowentry/*, /p4/*	256
ryu_flow_stats.py	/stats/flow/<dpid>, string-action form	190
kernel_notifier.py	Pushes switch and link events north	144

Current file sizes, not diff counts.

sFlow synthesis, and how it was proven

bmv2 emits no sFlow of its own — so the proxy manufactures it, byte-compatible with what OVS sends

1 Valid sFlow was not enough

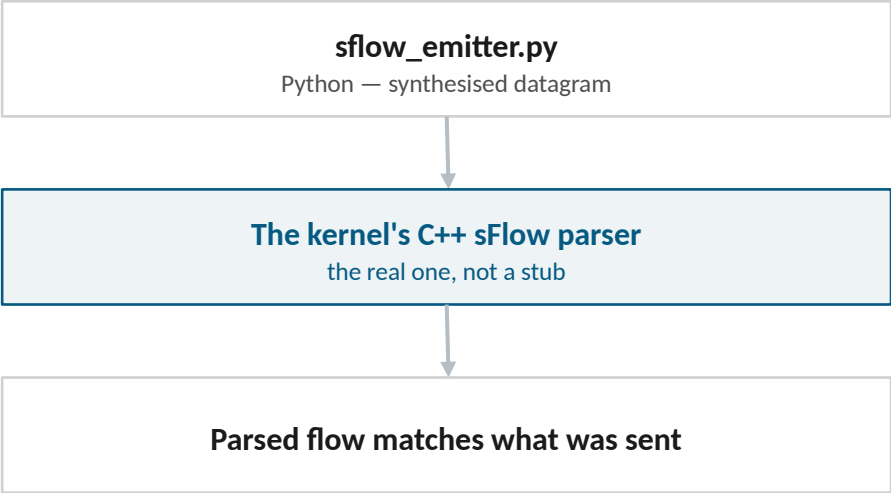
The kernel's decoder is a hand-rolled fixed-word-offset parser, not a general sFlow library. The datagram must have the same shape OVS produces — two flow records, extended_switch then raw header.

2 Agent addresses come from the same file

The kernel attributes a sample to an edge by AgentKey{agentIP, port}. An address it does not recognise yields telemetry attributed to nothing, with no error — so the emitter reads the topology JSON the kernel reads.

3 Proven by cross-language round trip

The Python emitter's real output is fed through the actual C++ parser the kernel uses, and separately compared byte for byte against datagrams captured from a working OVS run.



Two independent checks

Round trip	test_SFlowEmitterRoundtrip.cpp
Byte compare	test_sflow_emitter.py vs OVS capture

The telemetry sample path

From a sampled packet in the pipeline to a flow the kernel can attribute

1 A PRE clone session, programmed over P4Runtime

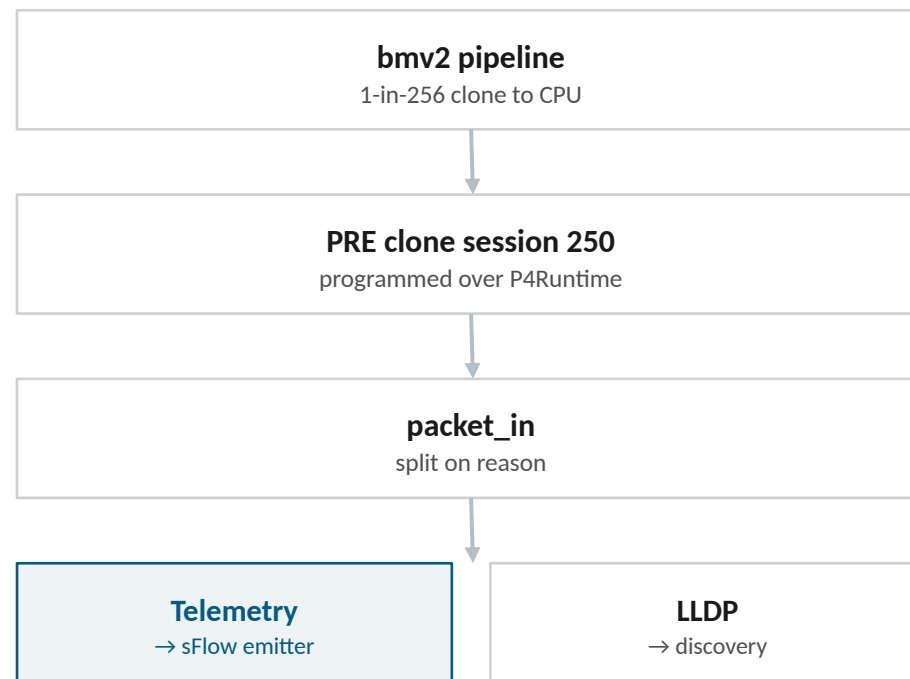
Session 250 is what the pipeline clones sampled packets into. It lives in the pipeline's PRE, so bmv2 rejects it if no pipeline is loaded — the proxy programs it only after it has pushed the config itself.

2 Samples and packet-ins share one channel

Both arrive as `packet_in`. They are separated by a reason field, and the split matters for load as well as correctness: sampling is 1-in-256 of all traffic, so letting samples reach the LLDP parser would bury discovery.

3 A trap worth naming

A third controller header compiles cleanly but is silently ignored — P4Runtime matches controller headers by name, and only knows `packet_in` and `packet_out`.

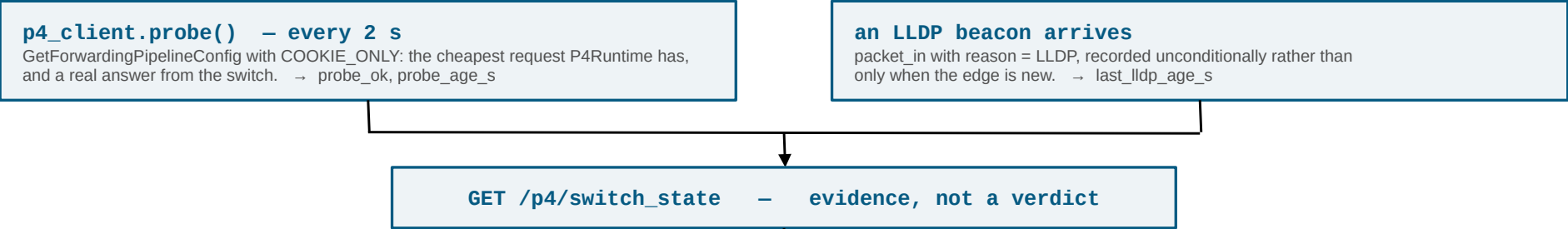


Verified on ten live bmv2 switches: 2,700 packets over ten hops produced 107 samples against a model prediction of 105.

Liveness: how the twin decides a bmv2 switch is alive

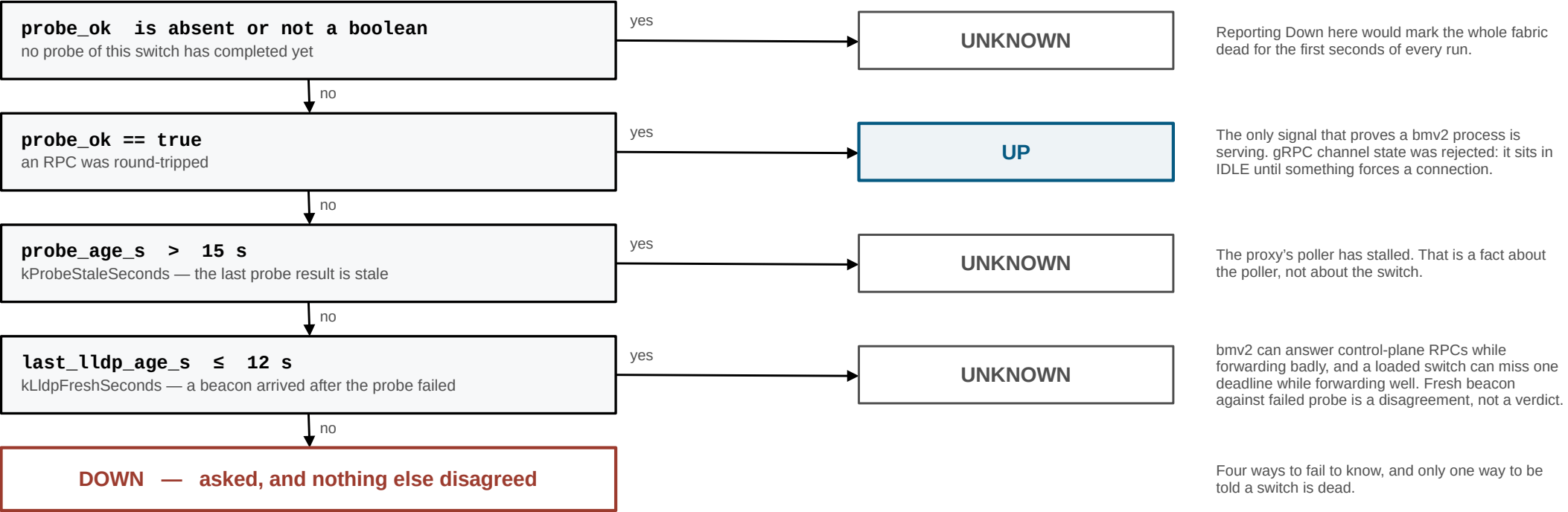
Two independent kinds of evidence, and a verdict that is allowed to come back “cannot tell”

GATHERED BY THE PROXY



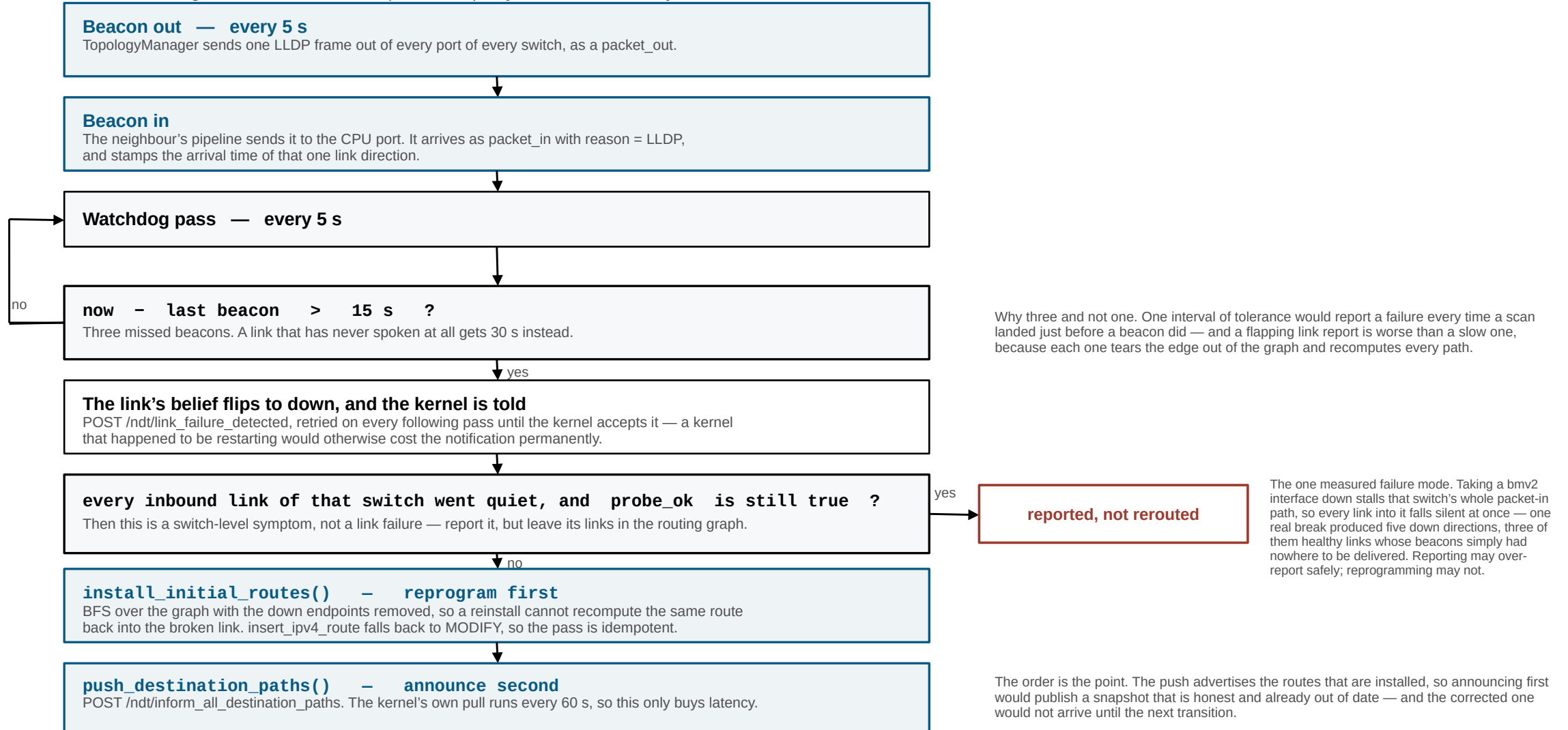
the kernel polls this and decides — nothing above this line changes

DECIDED BY THE KERNEL



Failover: what turns a missing beacon into a new route

bmv2 raises no link-down signal, so the absence of a packet the proxy sent itself is the only evidence there is



Phase 7: switch power management

A twin has to be able to switch a node off and take it back again — and the order those steps land in is part of the mechanism

1 Power off, and keep enough to come back

The ndtwin-p4-power helper shuts a bmv2 switch down entirely while preserving the state needed to restart it. A switch that is off disappears from /v1.0/topology/switches rather than staying in the list pretending to exist.

2 Re-adoption after a restart

POST /p4/readopt/{dpid} rebuilds mastership, the pipeline, the clone session and the routes. It is one call because those four have to happen in that order, and a partial re-adoption is worse than none.

3 A known 30-second window, stated plainly

Re-adoption reports "adopted, routes not yet installed" while the links are still being rediscovered by LLDP. The gap is expected and observable rather than hidden.

Live acceptance

The switch was held off for	135 s
From power-on to fully re-adopted	1.50 s
The graph was sampled at 10 Hz for	678 readings
Readings that wrongly showed it up	0

Re-adoption is one call, not four, because mastership, pipeline, clone session and routes have to land in that order — and the 10 Hz sampling is there to prove the graph never flickered up in between.

SECTION 2

Test tooling and documentation

The five-layer harness, what each layer gates, and what was written down

The methodology is the real technology

Four ideas, each one replacing a judgement call with something that can fail

1 Differential testing

The OVS path is known-good, so its behaviour is the specification the P4 path is checked against. No separate spec to write, and no argument about what correct means.

The same method timed the inherited router at $180.75 \text{ s} \times 3$ against 51.8 s now
— no claim about anyone's code required.

3 Allowlist gates

Any warning or OVS/P4 difference not on a list is a failure. Every accepted difference has to be written down as a sentence with a reason.

New problems cannot hide inside old noise.

2 Mutation testing as an acceptance gate

A test does not count until it has been watched failing: break the implementation, see it go red, restore. The evidence is filed per test.

This caught eleven tests that passed while proving nothing at all.

4 Evidence-based design

Liveness has three states, not two. Southbound calls return `OpResult`, not `bool`. "I could not tell" and "it failed" are never allowed to render as "it worked".

Liveness on page 16 is the instance of this that carries the most weight.

Five layers, five different questions

Different failure modes surface at different stages, and each layer costs a different amount to run and to debug

Layer	The question it answers	Cost	Mininet
L0 Build check	Does every component still compile?	seconds–minutes	no
L1 Unit tests	Is unit behaviour correct — and still correct in a shared process?	~2 minutes	no
L2 API contract	Does /ndt/* honour its structure, invariants and error paths?	needs a running stack	yes
L3 Component contract	Change one endpoint — which of the seven components break?	needs a running stack	yes
L4 Differential	Does the P4 path behave like the known-good OVS path?	full stack + traffic	yes

What the suite contains, and what makes it trustworthy

A test count proves nothing on its own — so every test here was watched failing before it was allowed to count

603

gtest cases
in 47 files

453

P4-proxy tests
in 17 files

243

kernel-side Python
in 7 files

7

shell suites
orchestration and gates

1 Every test ships with the mutation that breaks it

Applied, observed failing, reverted — and the evidence filed in four dated records under doc/audit/. This is what caught eleven tests that passed while proving nothing.

2 The tooling's own false passes were fixed

Four of them. unittest counts skipped tests inside "Ran N", so a fully-skipped file printed green — and three tests sat below unittest.main(), so they had never executed at all.

3 Log output is a pass/fail gate, not a thing to read

check_logs.py fails on any warning or error not on the allowlist, has a FORBID list that overrides it, and treats crash signatures as unconditional failures that no allowlist can excuse.

Why the count moves

426 → 547 → 603 in seven days. That is real growth, not miscounting: three live defects each got a regression test, and four new tool suites arrived with their own.

The claim being made

Not "we have many tests". The claim is that each one has been observed failing for the reason it exists — which is a different and checkable statement.

What was written down

Twenty-four documents under doc/, organised so that the next person has a route through

Operational — how to run it

- full_test_runbook
- the whole stack, end to end
- ovs_manual_test_runbook
- the OVS round
- p4_manual_test_runbook
- the P4 round
- environment_gotchas
- sudo, pgrep, cleanup, ordering
- testing-manual
- the current consolidated guide

Design and status

- p4_bmv2_support_plan
- phase-by-phase plan and status
- phase7_power_mechanism_design
- the power path
- ndt_api
- every /ndt/ endpoint, with byte-order and lock-type clarifications
- HANDOFF
- what is still open

Investigation records

- p4runtime-mastership-spec-check
- the conclusion that was withdrawn
- bm2-performance-report
- the throughput A/B
- src-ip-endianness-review
- a field that looked wrong and was not
- doc/audit/
- wedge trace, mutation evidence, scoped reviews

The one that matters most is the one listing what is not done

test_coverage_gaps records what has no test yet, and HANDOFF records what is still open — including a defect found and deliberately left unfixed. A handover document that only lists successes is not a handover document.

SECTION 3

Measured results

Differential testing, failover, load, twin accuracy — and a live demo

The P4 path matches the OVS baseline

L4 differential: the OVS path is the specification, and every difference the P4 path shows has to be justified in writing or it fails

PASS	18	0	12
L4 differential	allowlist entries, each with a reason	unexplained differences	allowlist entries deleted once P4 caught up

1 The allowlist is the mechanism, not a formality

compare_baseline.py fails on any difference not listed. Each entry is an endpoint, a regex and a reason, so "P4 cannot do this yet" has to be written as a sentence.

2 Entries are deleted as phases land

Twelve were removed once P4 implemented what they excused — each re-verified against the live system first, not just crossed off.

Why this is the strongest single result

It is the only check that compares two independent implementations of the same behaviour. Unit tests confirm the code does what I thought it should; this confirms the P4 path does what the known-good path does — a claim I could not have written down myself.

PASS was recorded at dac192b and is stated as of that commit, not as a live status.

TO ADD comparison output, difference table

Failover, measured under controlled conditions

One fault model, one script, four cells, forty runs — the outage is timed from a continuous ping, not inferred from logs

Cell	n	Mean	SD	Range
P4 / 4 hosts	10	13.68 s	1.6	11.0 – 16.4
OVS / 4 hosts	10	15.71 s	1.5	13.7 – 18.1
P4 / 128 hosts	10	16.59 s	2.0	14.3 – 20.8
OVS / 128 hosts	10	51.75 s	3.3	47.0 – 56.4

What makes these hold

Continuous timestamped ping at 5 packets/s, so the outage is measured rather than inferred; the link to break is resolved at run time from the live path; netem re-checked every 5 s; recovery counted only while the fault is still present. “Recovered fully” is a filter rather than an impression — the raster only accepts a run with exactly one gap longer than a second, and all forty qualified.

128 hosts was assumed to be too heavy

A comment in the topology script said bmv2 might not take it, and that assumption had gone unchecked. It takes it — 10/10 switches up, 16,256 paths converged in 11 s. What actually blocked the cell was four hard-coded four-host lists in three files.

At 4 hosts P4 is 2.0 s faster — 13%. Welch t = 2.89, p = 0.0098, 95% CI 0.55 – 3.50 s. At 128 hosts the gap is 3.12× and the two ranges do not overlap at all. Every one of the forty runs broke once and then recovered fully. (9467ea0 / b6b75fa / 213d209)

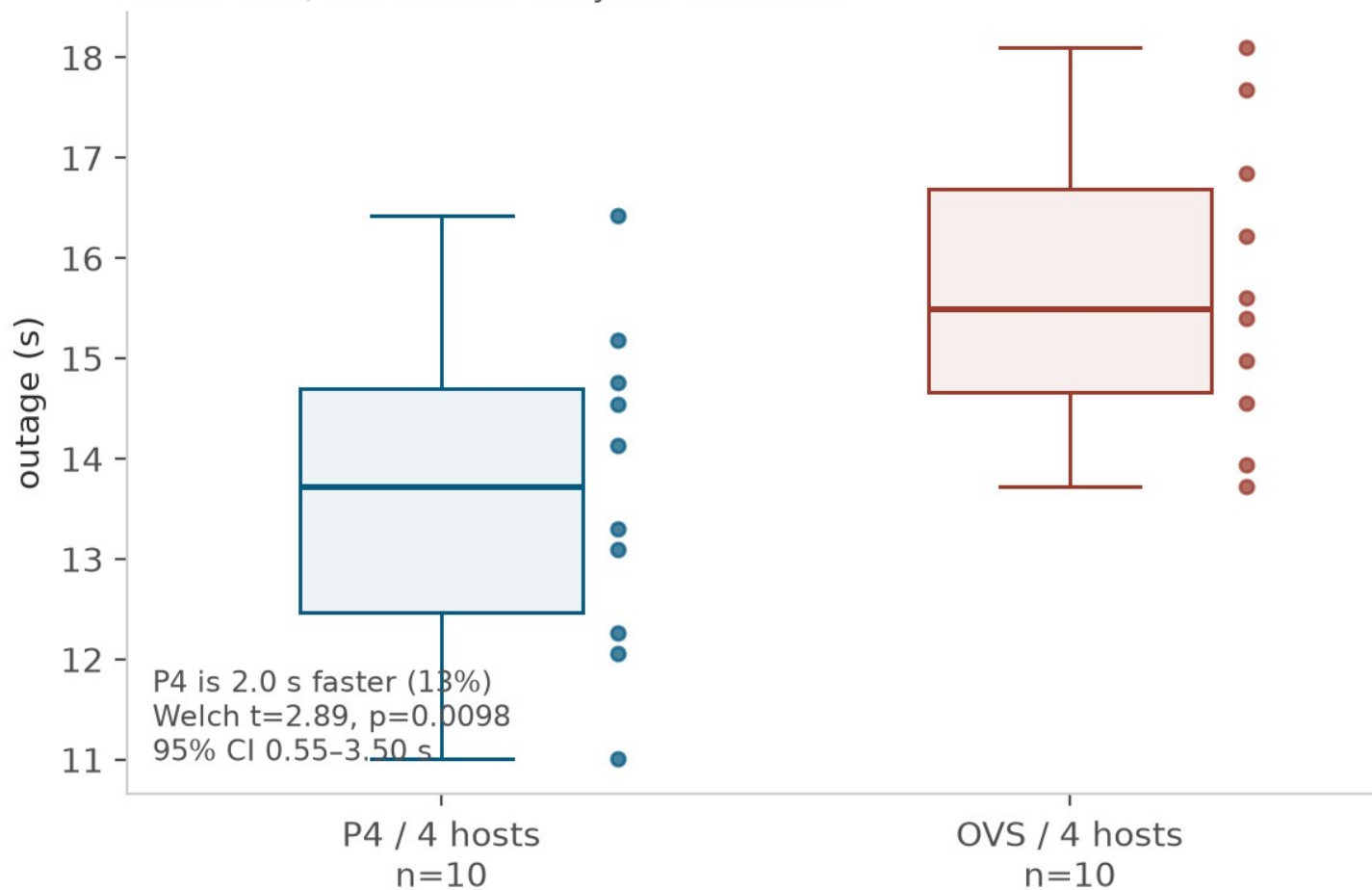
What separating the variables shows

Scaling 4 → 128 hosts costs OVS 3.29× but P4 only 1.21×, so the interaction is larger than either main effect. "The data plane is the smallest of the three" is true at 4 hosts and false at 128 — the mechanism is not established, and the honest statement is that the two control paths degrade differently with scale.

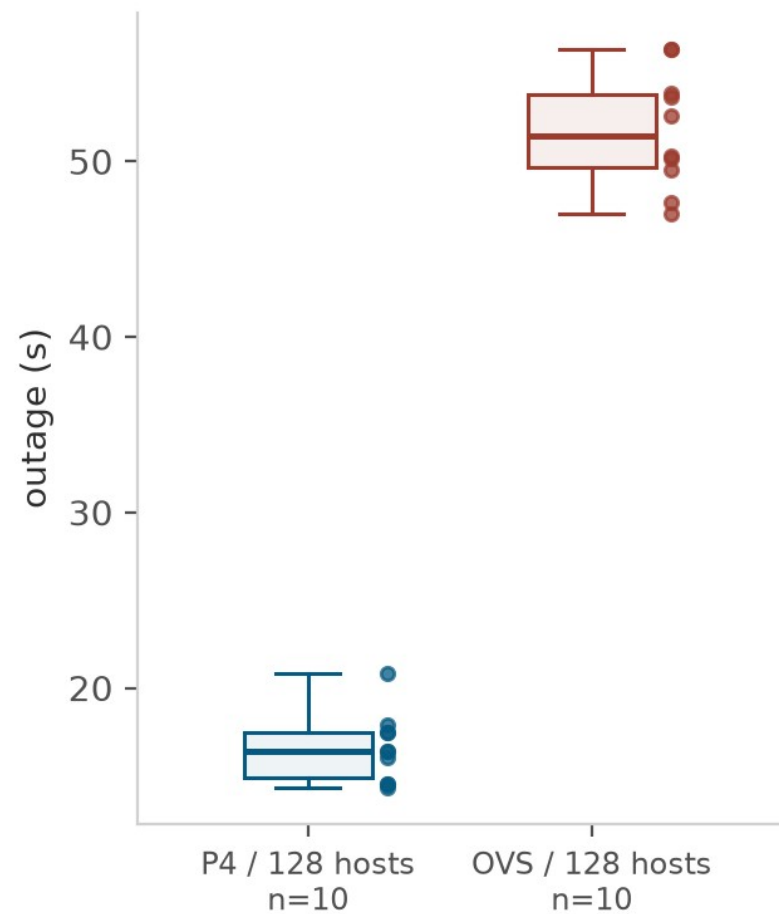
Single fault, single host pair, single machine. Quote the interval, not the mean.

Failover on a matched topology

40 live runs, one method. Every run recovered.

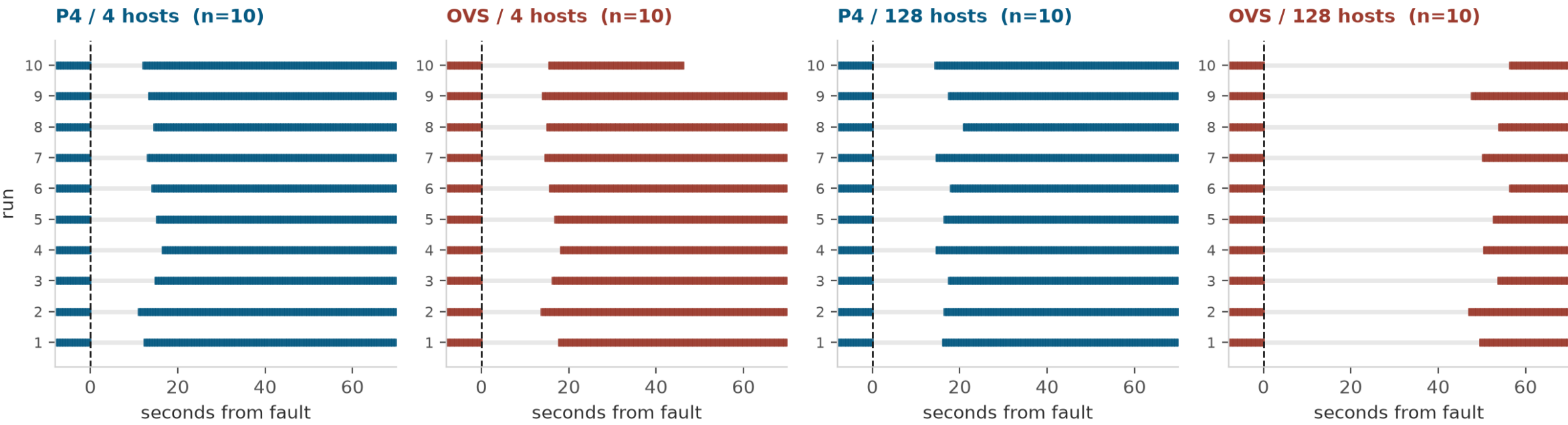


At 128 hosts the gap is 3x



Every ping of every run: one clean outage, then full recovery

Each tick is a reply; the grey bar is the outage. Aligned on the last reply before the fault. No run shows a second loss or a flapping tail.

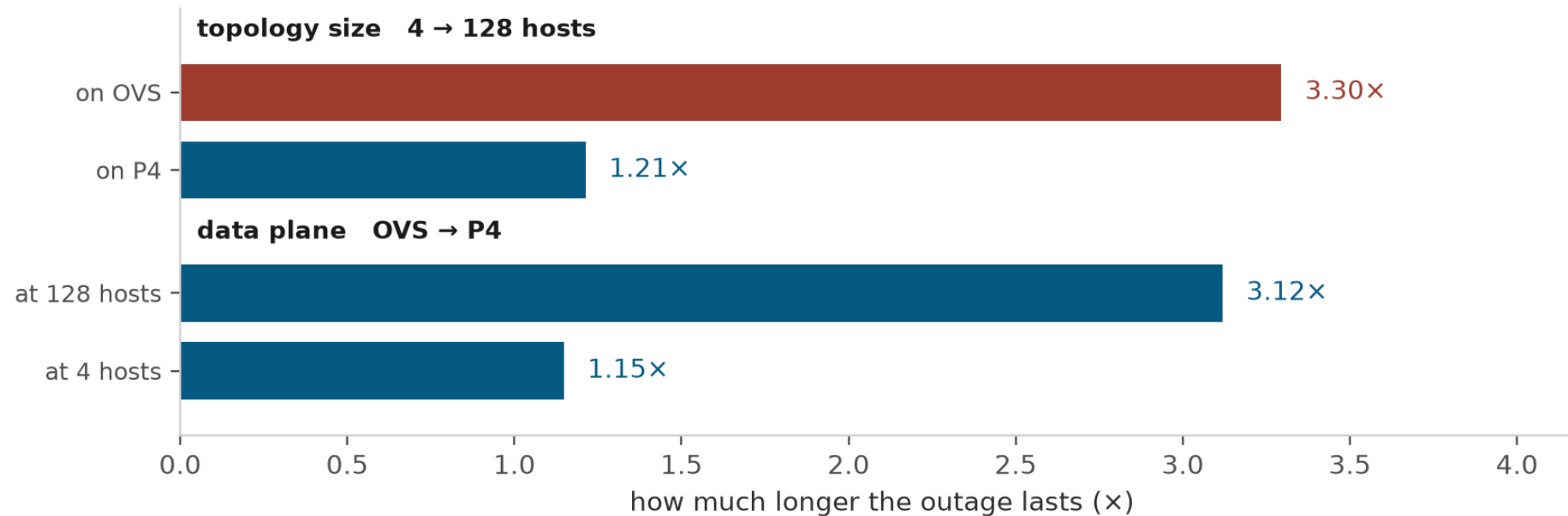


Three things change how long a link failure lasts — one changes whether it ends

Same fault, same method, $n=10$ per cell. The first is a difference in kind. The other two are multipliers — but neither has a single value: each one depends on the level of the other.

inherited router (since fixed)  **never recovers**

Measured $180.75\text{ s} \times 3$ with the fault held 180 s — it came back only when the fault was lifted. Not a slower recovery: no recovery.



What the data planes can actually carry

Same fabric, same 3-hop path, same iperf3 script — two bmv2 builds and an OVS control

Build	Configuration	UDP delivered	TCP goodput	Packet rate
bmv2 — stock build	-O0, all logging	~40 Mbps	24.2 Mbps	~3.6k pps
bmv2 — fast rebuild	-O3, logging macros disabled	~460–530 Mbps	431 Mbps	~50.8k pps
OVS — control	shaped by TCLink at 1 G	980 Mbps	—	—

1 The ceiling is packets, not bits

bmv2 is an interpreted reference implementation; at -O0 with logging macros the per-packet fixed cost dominates. 64-byte and 1400-byte frames hit the same packet rate, which is what identifies the limit.

2 The loss is invisible to interface counters

It happens in the first on-path switch's input buffer. h1 sent 89,296, s1-eth3 received 89,296 — and s1-eth1 transmitted 33,456. Nothing in between reports a drop.

The published figure describes neither build

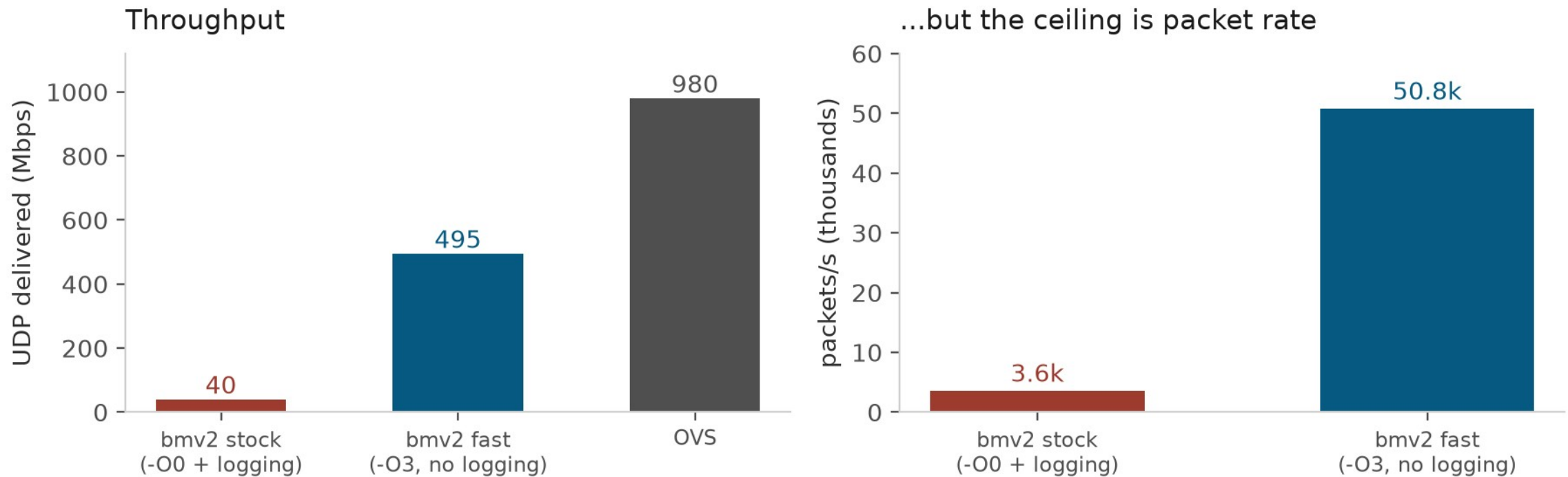
~170 Mbps [2] is 4× above the stock build and ~3× below the fast one, so it is cited as an order of magnitude and nothing more.

What it costs the twin

At 1 Gbps declared, single-flow utilisation caps at ~4% on stock and ~43–53% on fast — so TE's 70% congestion threshold is physically unreachable on stock.

Two bmv2 builds on the same fabric and script

Same pps at 64 B and 1400 B: the cost is per packet, not per bit. 12-18× from the build alone.



210 Mbps sustained, then a link broken underneath it

The earlier runbook only ever loaded one flow at 10 Mbps, and only broke links while idle

210 Mbps	288 / 288	0	2.7% / 0%
carried, four concurrent flows, five minutes	the graph never moved	spurious link events from Ryu	loss on the broken flow / the other three

1 Rerouting does not disturb traffic it is not rerouting

Breaking s1-eth2 under load cost 35,445 of 1,337,473 datagrams on the flow using it, and exactly zero on the other three — including the one sharing two switches with it.

2 The link-usage metric tracked the load correctly

About 5× for 20× the traffic, which is what the metric's own definition predicts: the denominator grows with the load.

3 One measurement points somewhere specific

The stale-path window reproduced at 14 s under load against 13 s idle — which points at a fixed cache period rather than anything load-dependent.

An old warning left standing

The runbook records 100 Mbps starving LLDP badly enough that Ryu deleted 19 healthy links. Not reproduced here — but this was four 50 Mbps flows on different paths at half the per-flow packet rate, which is a different condition. The warning stays until someone reruns it as written.

How accurate the twin's link usage is

Accuracy is not a number here but a curve — the estimate is unbiased, and its spread is the sampling-theory floor

Window	200 Mbit/s ±%	theory	20 Mbit/s ±%	theory
1 s	20.7	23.9	70.9	75.6
10 s	8.1	7.6	25.7	23.9
60 s	3.1	3.1	6.8	9.8
150 s	1.9	2.0	4.9	6.2
430 s	1.2	1.2	—	—

1 Unbiased across 430× of window and 10× of load

Median ratio 0.97 – 1.02 at every window and both loads. Over 3,599 samples the mean reads 205.5 Mbit/s against a true 205.8 — 0.15% out.

2 The resolution floor is one sample

quantum = 256 × frame bytes × 8 — per-flow, not a constant: four runs gave 1490, 1490, 1494, 1506 bytes.

The sentence that survives questioning

Not "we are accurate to X%" — that number does not exist. The same instrument is ±21% at a one-second window and ±1.2% at 430 s. What can be claimed is: unbiased, with precision set by sample count, exactly as sampling theory predicts [1].

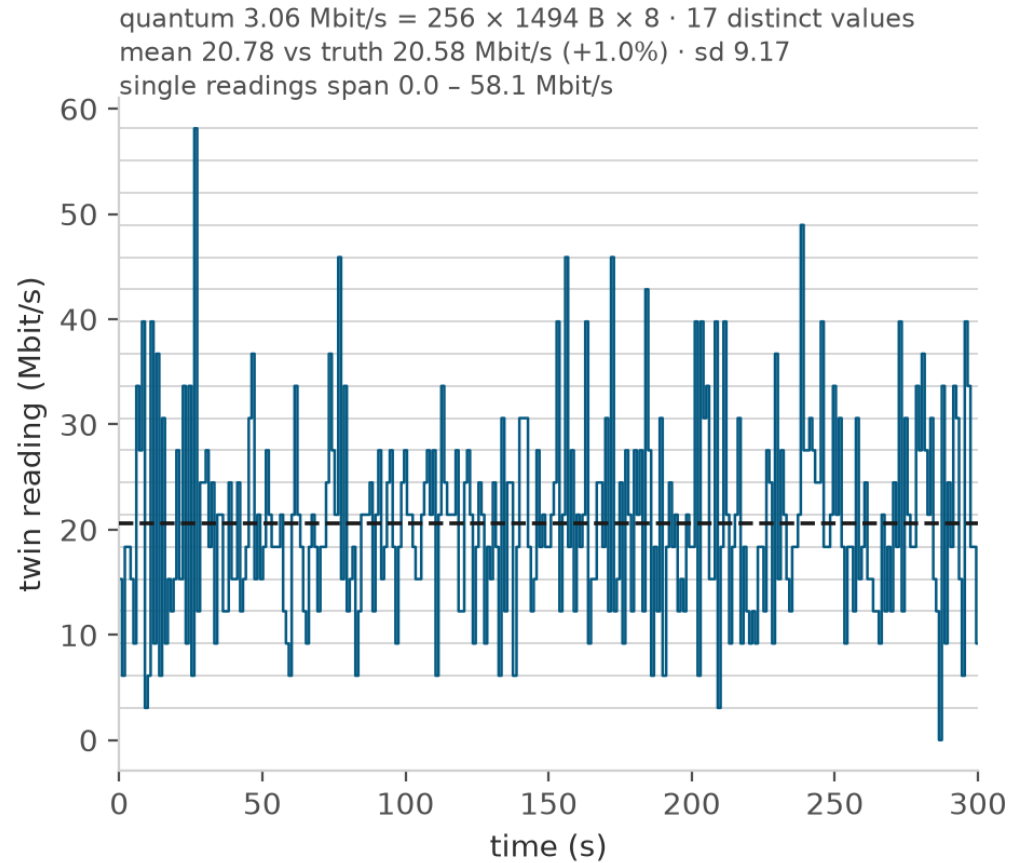
Where it stops working

On a 1 Gbps link a flow below about 3 Mbit/s is invisible and utilization_percent moves in 0.31% steps. A single one-second reading at 20 Mbit/s ranged from 0.15× to 2.38× of truth. And the four-host OVS cell has no sFlow configured at all: it reads zero, with status success.

The resolution floor is one sample — on both data planes

Grey lines are the only values the twin can report. The frame length differs per flow, so the quantum does too — it is not a constant.

OVS — native sampling, 20 Mbit/s



P4 / bmv2 — synthesised, 20 Mbit/s

quantum 3.05 Mbit/s = $256 \times 1490 \text{ B} \times 8 \cdot 16$ distinct values
mean 20.72 vs truth 20.58 Mbit/s (+0.7%) · sd 8.06
single readings span 3.1 – 48.8 Mbit/s

time (s)

Synthesised telemetry against native

bmv2 emits no sFlow of its own, so the proxy manufactures it — this measures whether a manufactured sample is as good as a real one

Window	P4 median	OVS median	P4 ±%	OVS ±%	theory
1 s	0.965	0.966	54.6	70.9	75.5
2 s	1.001	0.967	47.5	51.3	53.4
5 s	1.008	0.989	29.6	35.0	33.8
10 s	1.016	1.022	20.1	25.7	23.9
30 s	1.008	0.990	8.8	11.8	13.8
60 s	1.003	1.007	6.3	6.8	9.7

1 Synthesised is as unbiased as native

Median ratio 0.97 – 1.02 at every window on both planes, each against its own measured ground truth of 20.6 Mbit/s. Both sample at 1-in-256, so the theory column is shared.

2 And it adds no error of its own

The P4 spread is at or below the sampling-theory floor at every window.

Measured at b6b75fa. P4 stock build, 20 Mbit/s, 600 s, 2,400 samples, none dropped.

One result I cannot explain yet

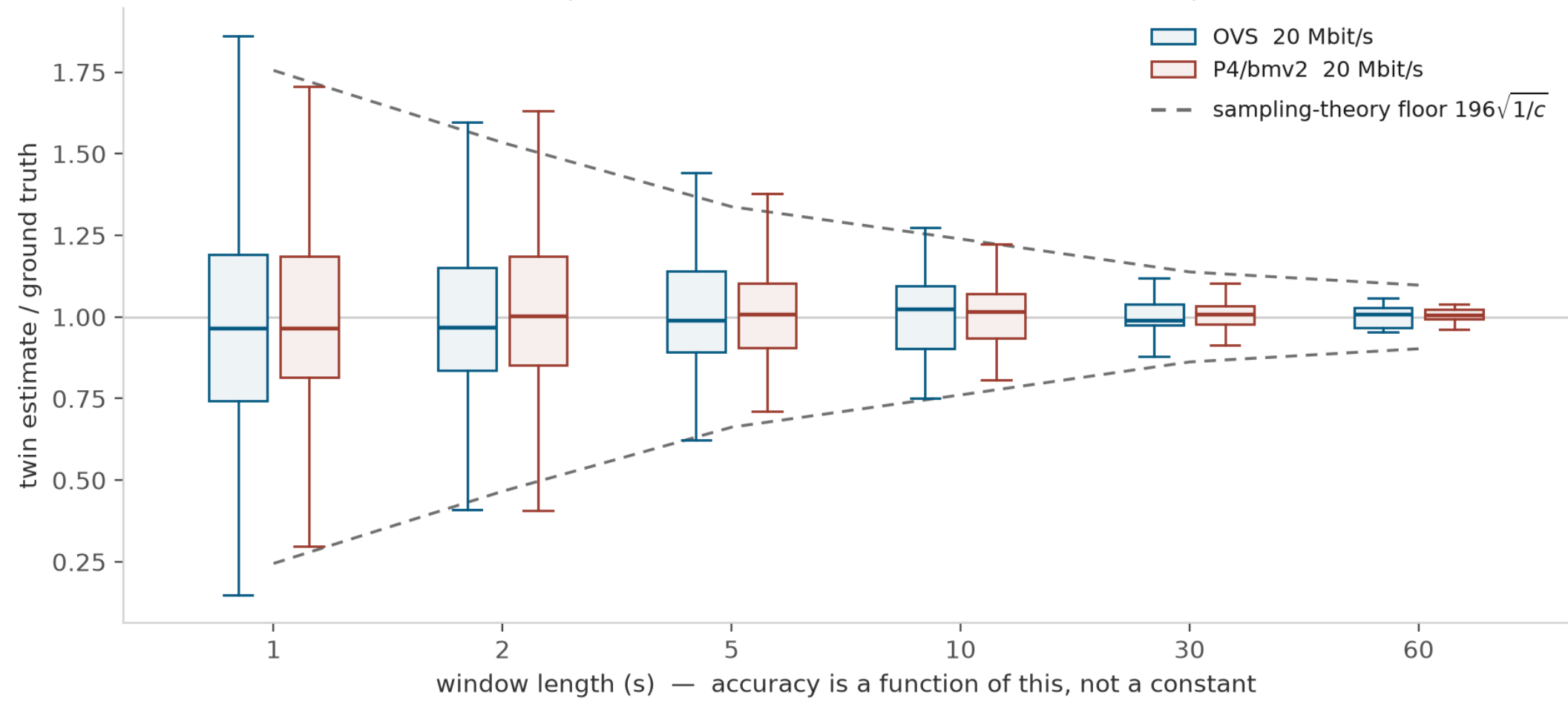
P4 sits consistently below the theoretical floor — 54.6% against 75.5% at one second — while OVS sits on it. Below the floor is not a problem, but the mechanism is unproven. One candidate is that bmv2's per-packet random() is not independent, making the sampling closer to systematic than Bernoulli. Reported as an observation, not a claim.

Why the two cells are comparable

Different host counts, but the same switch core — ten switches, thirty-two inter-switch links — and the same 1-in-256 rate. Sampling precision depends on the samples taken on the link, not on how many hosts hang off it. The per-hop figure therefore shows four hops on OVS and two on P4: the flow is shorter in the smaller layout, and each panel is an internal comparison.

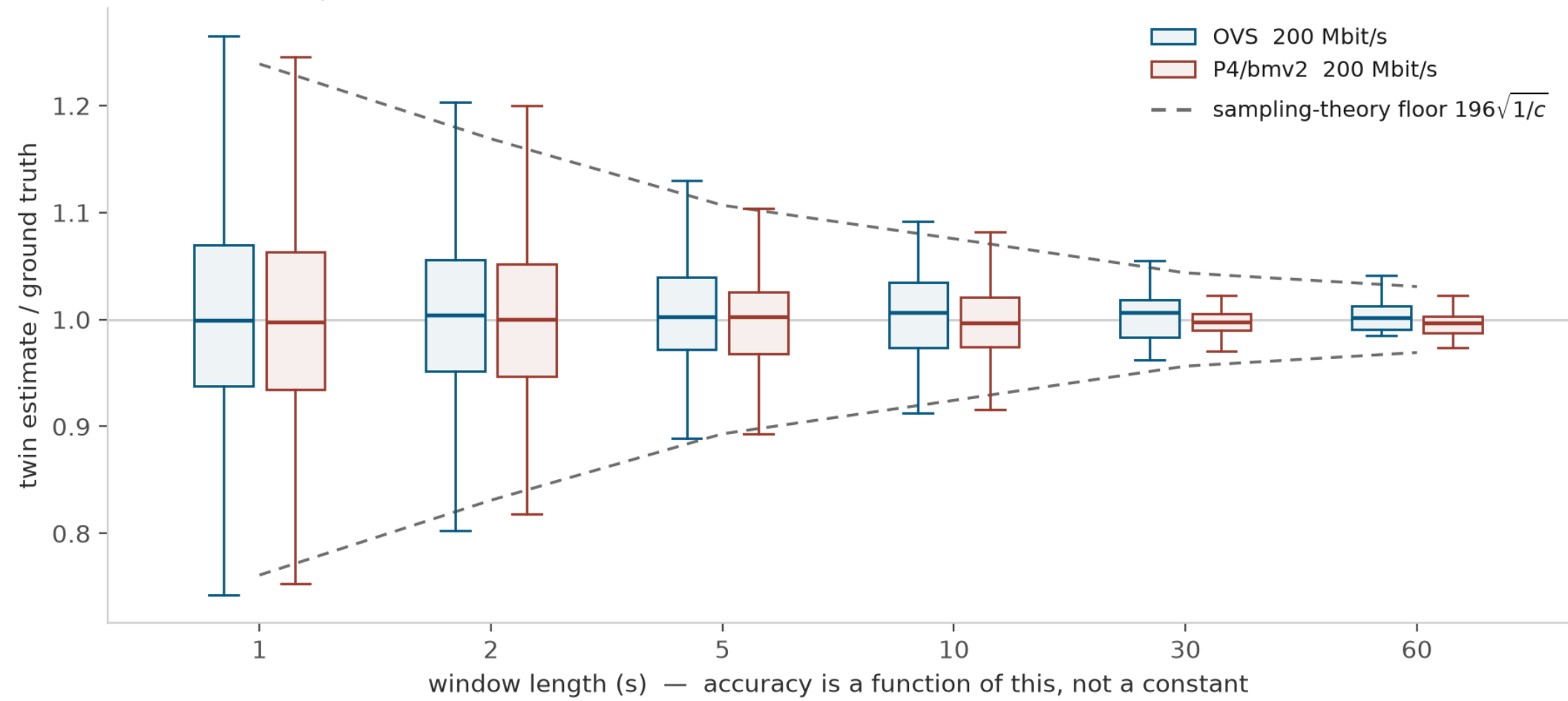
Telemetry accuracy: synthesised (P4) vs native (OVS) sampling

One fixed-rate flow, 20 Mbit/s. Boxes = spread of window estimates; the twin is unbiased on both planes.



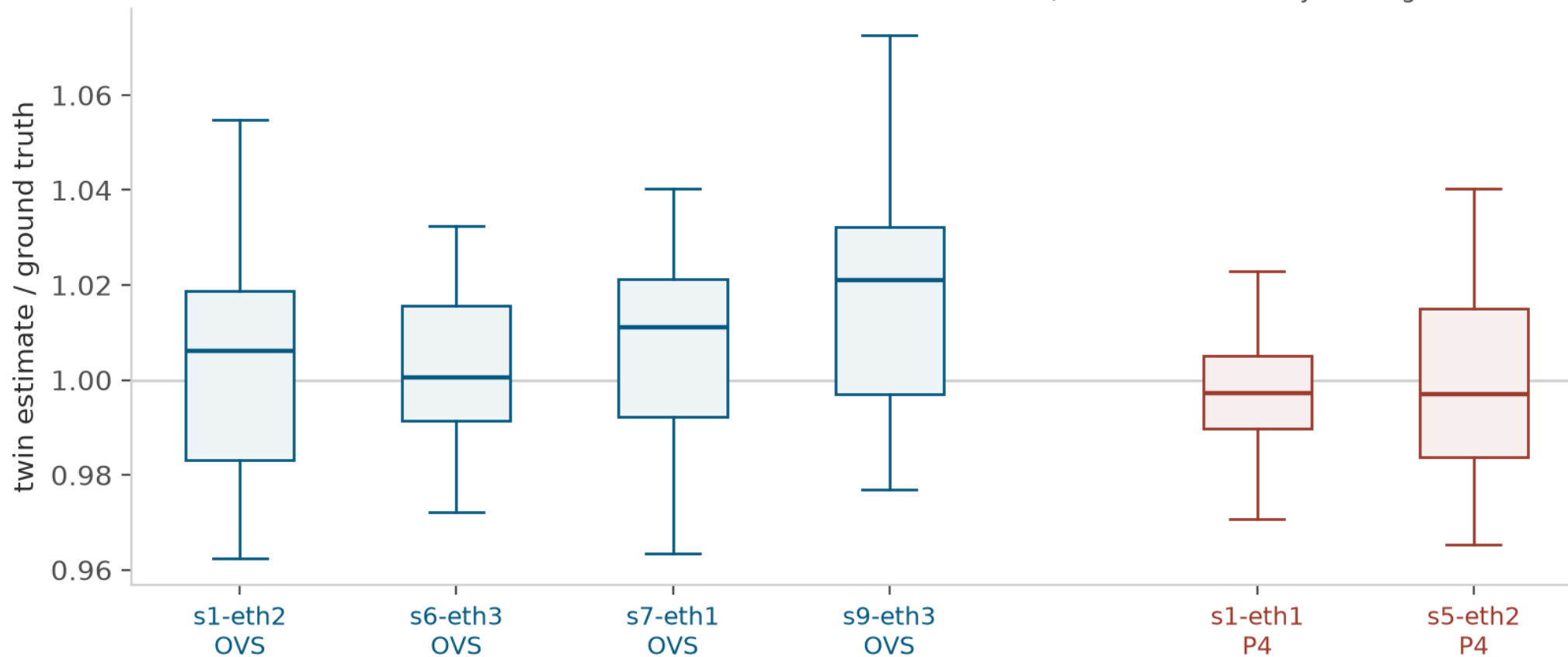
Telemetry accuracy at 200 Mbit/s: synthesised (P4) vs native (OVS)

Same load on both planes; P4 needed the -O3 bmv2 build to reach it.



Every hop on the path, same flow, same window

30 s windows. The flow crosses 4 links on the OVS cell and 2 on the P4 cell; the other links carry nothing and are not shown.



An independent pass over the running system

Read-only checks against the live OVS stack, run from a perspective that did not know the implementation

51	12	1	11
read-only checks against the live stack	findings raised	real defect — fixed	documented behaviour or false positives, each adjudicated

1 Every finding was adjudicated, including the wrong ones

Eleven of twelve turned out to be documented behaviour or false leads. Each was run to ground and the verdict written down, because an unadjudicated finding is indistinguishable from an ignored one.

2 A separate pass over unreviewed findings

Forty-seven high-severity items produced twenty-one real problems. That ratio is worth stating rather than reporting only the hits.

Why an outside perspective was worth arranging

Everything else in this deck was verified by the person who wrote the code. That is a real limit, and the only way past it is to have something check the system without knowing what it is supposed to do.

Demo

1

Link failure and reroute

Break a single link, watch the twin detect it, reroute, and read the rewritten rule back over P4Runtime. Roughly 30 seconds end to end.

TO ADD screen capture

2

The P4 fabric in the Web GUI

The same interface the lab already uses, driven by a data plane it was never written for — topology, flows and link usage, all through the unchanged /ndt/ API. The GUI is read-only, so anything that changes state is done from the Mininet CLI beside it.

TO ADD screen capture

3

Liveness moving through three states

Kill a switch and watch Up become Unknown before it becomes Down. Unknown is held while the LLDP beacon is still fresh and only becomes Down when it expires — so one failed probe never rewrites the topology.

TO ADD screen capture

Recorded, not performed

All three are procedures already written down and already executed on the live stack, each with its own recording runbook. What is shown is a replay of a recorded run rather than a new experiment carried out in front of an audience — the failure modes here are timing-sensitive, and a live attempt would be demonstrating luck rather than the system.

Where it stands

What shipped, what is deliberately unfinished, and what is simply unknown

Delivered

A second data plane, with nothing above it changed

P4/bmv2 fully attached — the kernel, all seven components and the Intent Translator are untouched.

The shared path made honest

The silent defects in the baseline are fixed, so the reference the P4 path is verified against can be trusted.

Both data planes have a decidable health standard

Five layers, differential comparison, and mutation evidence per test.

The queued-endpoint gap is worse on OVS than on P4

Same rule, same 200 queued, both data planes. P4 writes "dispatched install failed"; OVS writes nothing at all — 16,324 log lines, zero errors, and the rule never installed. On OVS the gap is not just unqueryable, it is untraceable after the fact.

Also worth asking about: a P4Runtime specification report was prepared and then disproved by its own final check before it was sent.

Open, and stated as open

Locks have no owner	any caller can release any other caller's lock — and seven components share them
Queued flow endpoints	200 queued, but nothing can ask whether the rule landed
Power-on can be a no-op	within ~10 s of shutdown it returns Success and does nothing
Proxy restart loses rules	every installed rule is dropped, and the twin reports full health
Phase 3	proxy endpoint completion — the only phase not started
Ryu wedge	root cause unproven after four falsified hypotheses
Election-id reuse	deliberately left; changing it means re-verifying re-adoption
popen("curl ...")	22 sites across 3 files, deferred on purpose
bmv2 stock throughput	~40 Mbps per switch — a design trade-off, not a defect

The four marked items were found in the last two rounds of live testing and are deliberately unfixed before this report. All seventeen open issues are collected in doc/KNOWN-ISSUES.md, ordered by whether normal operation reaches them.

Future work

Three directions, at three different levels — a design problem, a structural problem, and an architectural one

1 Concurrency control has two blocked routes, and both are located

The pessimistic route: `release_lock` only checks that a lock exists, never who holds it — a second connection with no credentials released a live lock and took it. Adding an owner is not sufficient on its own: `LockManager::renew` checks `isLocked` but not expiry, so an already-expired lease can be renewed indefinitely and TTL stops meaning anything when a holder dies.

Next: an owner token plus a `now < expiry` guard. The optimistic route stays blocked until commit failures are detectable at all.

2 `topology_manager.py` carries three responsibilities in one class

1,516 lines covering graph and routing, LLDP and liveness, and rule translation. The evidence is already in the file: two mutually independent locks, one guarding the graph and one the liveness bookkeeping, with a comment stating they are never held at once.

Next: split along the line the two locks already draw. A class that needs two unrelated locks is usually two classes.

3 Asynchronous dispatch needs a completion handle

`processFlowBatch` returns 200 queued and the dispatcher sends afterwards, so a switch's rejection has no route back to the application. Both applications that write flows discard the response, so returning 202 instead would change nothing.

Next: a queryable job id or a synchronous path — a contract change across seven components, which is why it is future work rather than a fix.

Four smaller ones are written down but not on this slide: per-flow 5-tuple rules end to end, LLDP timer tuning judged on false-positive rate rather than detection time, telemetry accuracy with several flows and the applications running, and P4 measured beyond a single fault.

Planned for the next report

Three measurements already scoped — each one has a number it has to beat, and a way of being wrong

1

Find where the OVS 128-host recovery time goes

OVS takes 51.75 s to recover on 128 hosts against P4's 16.59 s, with no overlap between the two ranges. The mechanism is not established, so the job is to attribute the time rather than explain it.

Path computation is already excluded: the same fabric converges all 16,256 paths in about 13 s of real work on the Ryu side and 11 s on the P4 side.

Done when detection, recomputation and rule installation add up to the 51.75 s that was measured.

2

Make failure detection faster without making it wrong

Detection is set by constants rather than by the network: the beacon interval is 5 s, and both the link timeout and the watchdog interval are derived from it. Three missed beacons before a link is called down.

The experiment is a sweep of the beacon interval at 5, 3, 2 and 1 s, measuring detection time and false positives at each.

Judged on the false-positive rate, not the detection time. Detection time is certain to fall; a flapping link costs a global path recomputation.

3

Promote the fast bmv2 build to the default

Same fabric, same script: the stock build carries ~40 Mbps at 3.6k pps, the -O3 build with logging macros disabled carries 460–530 Mbps at 50.8k pps. The seam that switches between them already exists; the default is still stock.

It unlocks two things that are currently unmeasurable: TE's 70% congestion threshold, which is physically unreachable on stock, and telemetry accuracy with several concurrent flows.

Done when the whole L0–L4 suite passes on the fast build — changing the switch binary changes the timing every layer depends on.

One thing to settle before tuning anything: the code comment says detection takes 15–20 s and the last live round measured 10.7–14 s. Tuning a timer we cannot yet predict would leave us unable to say what moved.

References

[1]	Phaal, P. & Panchen, S. <i>Packet Sampling Basics</i> . sFlow.org.	The $\pm 196\sqrt{(1/c)}$ floor used on the two accuracy slides
[1a]	Jedwab, J., Phaal, P. & Pinna, B. <i>Traffic estimation for the largest sources on a network, using packet sampling with limited storage</i> . HP Labs HPL-92-35, 1992.	The binomial argument [1] rests on
[2]	Chen, Hu & Jin. <i>Performance Evaluation of P4 Programmable Switches in Emulation</i> . ACM SIGSIM-PADS '23. doi:10.1145/3573900.3591120	The published bmv2 throughput figure
[3]	p4lang/behavioral-model, <i>docs/performance.md</i> — ~1047 Mbps / 80k pps for one simple_switch.	The upstream baseline for the A/B
[4]	Zhou, Yang, Duan et al. <i>Network Digital Twin: Concepts and Reference Architecture</i> . IRTF NMRG, draft-irtf-nmrg-network-digital-twin-arch-07, 2024.	The definition of the term, and the scalability caveat
[5]	faucetsdn/ryu, <i>README</i> — “The Ryu project needs new maintainers”, pointing to OpenStack os-ken.	Southbound dependency risk

Where the 196 actually comes from

It is not a result from a paper. [1] is a technical page with no publication date that gives no citation for the expression itself; the mathematics underneath is the normal approximation to a binomial, and $196 = 1.96 \times 100$, the 95% z value. Phaal is an author of both [1] and [1a].